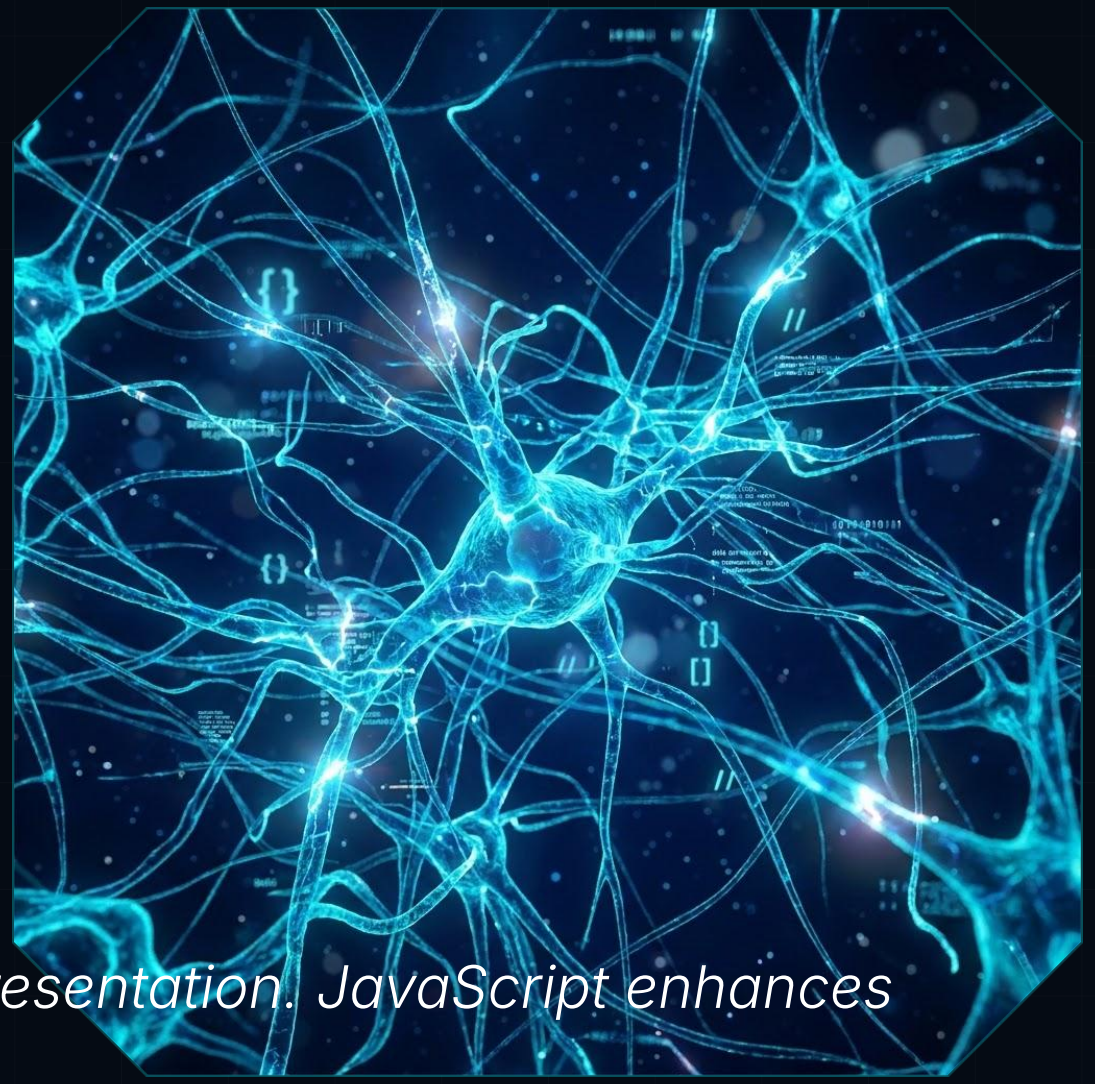
Lab 2: Modern React Architecture, Virtual DOM & State Machines

Semantic HTML, Modern CSS Layouts, Vanilla JavaScript & Agentic Workflow

MSc. Trần Vĩnh Khiêm
University of Information Technology, VNUHCM



> *"HTML is the foundation. CSS enhances presentation. JavaScript enhances behavior. AI enhances the developer."*

PEDAGOGICAL MANIFESTO

Engineering Standards in the AI Era: Generate & Verify

Embracing AI capabilities while holding ultimate engineering accountability.

Foundational Principles

- HTML is the semantic foundation; CSS handles presentation.
- JavaScript governs reactive state & computational behavior.
- AI enhances developer velocity, NOT replaces foundational comprehension.
- AI generates probabilistic code; Developers provide deterministic verification.
- Zero Monolithic Vibe Coding: Enforce atomic task decomposition and contract-first workflows.

engineering-contract.ts

```
// Engineering Verification Rule:
// Never accept unverified AI outputs into production.
export interface SystemContract<TState, TEvent> {
  readonly state: Readonly<TState>;
  transition(event: TEvent): TState;
  verifyInvariants(): boolean; // Must pass in runtime
}

// Verified Engineering Workflow:
// 1. Functional Slice -> 2. Type Contract
// 3. Atomic Gen       -> 4. DevTools / Unit Audit
```

DEPRECATION AUDIT

Modernization Audit: Eliminating Legacy Web Anti-Patterns

Purging obsolete specifications and aligning with Baseline Widely Available standards.

Deprecated Patterns (Strictly Forbidden)

- BANNED: 'var' scoping & implicit globals -> MUST USE 'const' & 'let'.
- BANNED: 'keypress' & 'e.keyCode' -> MUST USE 'keydown' & 'e.key' / 'e.code'.
- BANNED: Raw 'innerHTML' injection -> MUST USE DOM APIs or sanitized text nodes (XSS Prevention).
- BANNED: Inline event handlers (onclick=...) -> MUST USE centralized Event Delegation.
- BANNED: Outdated Babel standalones -> MUST USE modern bundlers (Vite/ESBuild) & JSX Transform.

legacy-vs-modern.js

```
// ❌ DEPRECATED & INSECURE (Legacy Lab)
function renderBad(data) {
  var div = document.getElementById('app');
  div.innerHTML = '<span onkeypress="doStuff(event.keyCode)">'
                + data.userInput + '</span>'; // XSS
Vulnerability!
}

// ✅ MODERN BASELINE SPECIFICATION (Lab 02)
export function renderModern(target, textContent) {
  const el = document.createElement('span');
  el.textContent = textContent; // Safe from injection
  el.addEventListener('keydown', (e) => {
    if (e.key === 'Enter') handleInput(e.target.value);
  });
  target.replaceChildren(el);
}
```

CURRICULUM ARCHITECTURE

The 3-Tier Knowledge Taxonomy: Core, Awareness, Later

Clear cognitive load management for professional software engineering.

Cognitive Stratification

• Tier 1: CORE (Must master and implement directly in-class)
- POJO VNode representation, custom createElement & recursive renderToDOM
- Cursor-based useState closures and event delegation hub.

• Tier 2: AWARENESS (Understand architecture for debugging/lookup)
- Babel AST compilation stages, automatic JSX runtime (_jsx)
- Fiber work loop, time slicing, SyntheticEvent pooling.

• Tier 3: LATER LABS (Enterprise production ecosystems)
- React Server Components (RSC), Suspense streaming, SSR hydration.

curriculum-matrix.json

```
{
  "Tier1_Core": {
    "VNode": "POJO with { type, props, children }",
    "State": "Linear closure cursor array",
    "Reconciliation": "O(n) type & key heuristics"
  },
  "Tier2_Awareness": {
    "AST": "Parser (Babylon) -> Traverser -> Generator",
    "Scheduler": "MessageChannel / requestIdleCallback"
  },
  "Tier3_LaterLabs": {
    "Next15_RSC": "Server Actions & Edge Runtime",
    "Hydration": "Selective Island Hydration"
  }
}
```

WORKFLOW PROTOCOL

Anti-Monolithic Vibe Coding: 4-Stage AI Decomposition

Banning single-shot prompt dumping. Enforcing atomic engineering discipline.

The 4-Stage Decomposition Pipeline

- Stage 1: Functional Slicing -> Break requirements into isolated sub-problems.
- Stage 2: Contract-First Specification -> Define TypeScript interfaces and schemas before generating code.
- Stage 3: Atomic Generation -> Prompt AI for exactly one isolated function/module at a time.
- Stage 4: Independent Verification -> Validate output via DevTools profiler, test cases, and git diff.
- Mandatory Artifacts: project-rules.md and TASK_DECOMPOSITION.md (WBS).

TASK_DECOMPOSITION.md

```
# Work Breakdown Structure (WBS)
- [ ] Task 1.1: Define VNode & Props interfaces (Contract)
- [ ] Task 1.2: Implement createTextElement factory
- [ ] Task 1.3: Implement createElement flattening & mapping
- [ ] Task 1.4: Implement recursive renderToDOM with typeguards
- [ ] Task 1.5: Atomic Git Commit: 'feat(core): implement vdom
engine'

* RULE: Never prompt AI with >1 sub-task simultaneously.
```

CORE SPECIFICATION

Deconstructing JSX: Syntactic Sugar & Abstraction Layers

JSX is not HTML embedded in JS; it is an independent AST macro language.

Syntax Extension Mechanics

- JSX is transpiled into declarative tree-construction function calls.
- HTML-like tags convey developer intent; compiler converts them to deterministic code.
- Tag distinction: Lowercase ('div') denotes native DOM string; Capitalized ('Card') denotes function reference.
- Attribute transpilation: Attributes become key-value pairs in the props object.
- Children nesting: Nested JSX nodes become variadic arguments or children arrays.

jsx-transpilation.jsx

```
// What the developer writes (JSX):
const ui = (
  <div className="profile-card" id="user-101">
    <h1>Engineer Profile</h1>
    <Badge role="Senior Lead" />
  </div>
);

// What the JavaScript Engine actually executes:
const uiTranspiled = createElement(
  'div',
  { className: 'profile-card', id: 'user-101' },
  createElement('h1', null, 'Engineer Profile'),
  createElement(Badge, { role: 'Senior Lead' })
);
```

COMPILATION PIPELINE

Under the Hood: AST Parsing, Transformation & Codegen

How Babel parses token streams into syntax trees and generates executable JS.

The 3 Stages of Compilation

- 1. Lexical Analysis & Parsing: Converts raw source string into tokens, building an Abstract Syntax Tree (AST).
- 2. AST Traversal & Transformation: Visitor pattern visits JSXElement nodes, applying babel-plugin-transform-react-jsx.
- 3. Code Generation: Traverses rewritten AST to emit high-performance JavaScript strings.
- Awareness Concept: Modern bundlers (Vite/esbuild/SWC) execute this pipeline in Rust/Go at native speeds.

ast-node-schema.json

```
// Simplified AST Representation of <h1 id="title">Hi</h1>
{
  "type": "JSXElement",
  "openingElement": {
    "name": { "type": "JSXIdentifier", "name": "h1" },
    "attributes": [
      {
        "type": "JSXAttribute",
        "name": "id",
        "value": { "type": "StringLiteral", "value": "title" }
      }
    ]
  },
  "children": [
    { "type": "JSXText", "value": "Hi" }
  ]
}
```

COMPILER RUNTIME

JSX Runtime Evolution: Classic vs Automatic Transform

Eliminating boilerplate and enabling compiler-driven bundle optimizations.

Key Differences

- Classic Runtime (pre-React 17): Requires 'import React from react' in every single file; emits React.createElement.
- Automatic Runtime: Compiler automatically imports '_jsx' / '_jsxs' from runtime packages.
- Memory Efficiency: Children passed in props directly; avoids array allocations for static children via _jsxs.
- Modern Standard: Universal support in Vite, Next.js, Babel, and TypeScript compiler (jsx: react-jsx).

runtimes-comparison.js

```
// 1. Classic Runtime Output (Needs React in scope):
import React from 'react';
React.createElement('div', { id: 'app' }, 'Hello');

// 2. Automatic Runtime Output (Injected by compiler):
import { jsx as _jsx } from 'react/jsx-runtime';
_jsx('div', { id: 'app', children: 'Hello' });

// Static children optimization with _jsxs:
import { jsxs as _jsxs } from 'react/jsx-runtime';
_jsxs('ul', { children: [
  _jsx('li', { children: '1' }),
  _jsx('li', { children: '2' })
]});
```

BROWSER INTERNALS

Browser Rendering Bottlenecks: Reflow & Layout Thrashing

Understanding why direct DOM manipulation degrades 60fps frame rates.

The High Cost of DOM Mutations

- Browser Pipeline: JavaScript -> Style Calculation -> Layout (Reflow) -> Paint -> Composite.
- Reflow Cost: Recalculating geometry of elements forces synchronous ancestor and sibling calculations.
- Layout Thrashing: Interleaving DOM reads (e.g. offsetHeight) and DOM writes causes catastrophic frame drops.
- The Solution: Virtual DOM batches updates in memory, isolating DOM mutations to a single commit phase.

layout-thrashing.js

```
// ❌ DISASTROUS: Interleaved Read/Write (Reflow Thrashing)
for (let i = 0; i < items.length; i++) {
  const width = container.offsetWidth; // FORCES LAYOUT REFLOW!
  items[i].style.width = (width / 2) + 'px'; // WRITES TO DOM
}

// ✅ VIRTUAL DOM STRATEGY:
// 1. Calculate all state mutations in pure memory (POJOs).
// 2. Compute minimal diff changeset.
// 3. Batch apply real DOM mutations in a single animation
frame.
```

ARCHITECTURE & DATA MODELS

Virtual DOM Architecture: Plain Old JavaScript Objects (POJOs)

Representing the UI hierarchy as lightweight, serializable data structures.

VNode Contract & Characteristics

- Lightweight POJO: No prototype chain, no heavy EventTarget properties (~200+ properties on real DOM).
- Contract: type (tag string or component function), props (attributes & handlers), children (array of VNodes).
- Deterministic Serialization: Enables server-side rendering, snapshot testing, and cross-platform native targets.
- Text Node Normalization: Leaf nodes represented consistently as TEXT_ELEMENT VNodes.

vnode-interface.ts

```
export type VNodeType = string | ((props: any) => VNode);

export interface VNode {
  readonly type: VNodeType;
  readonly props: {
    [key: string]: any;
    children: VNode[];
    nodeValue?: string; // For text elements
  };
}

// Sample VNode Instance:
const sampleVNode: VNode = {
  type: 'section',
  props: {
    className: 'container',
    children: [{ type: 'TEXT_ELEMENT', props: { nodeValue:
'Hello', children: [] } }]
  }
};
```

CORE ENGINE IMPLEMENTATION

Engine Implementation: createElement & Text Normalization

Constructing the immutable tree with proper array flattening and primitive wrapping.

Algorithmic Walkthrough

- Normalizing primitives: Wrap raw strings/numbers in TEXT_ELEMENT nodes to enforce structural uniformity.
- Variadic children handling: Use Array.prototype.flat() to handle nested JSX arrays seamlessly.
- Default props fallback: Ensure props is always a valid object even when null is passed.
- Immutability: Return newly constructed frozen objects to avoid shared state mutations.

create-element.js

```
export function createTextElement(text) {
  return {
    type: 'TEXT_ELEMENT',
    props: { nodeValue: String(text), children: [] }
  };
}

export function createElement(type, props, ...children) {
  return {
    type,
    props: {
      ...(props || {}),
      children: children
        .flat(Infinity)
        .filter((c) => c !== null && c !== undefined && c !==
false)
        .map((child) =>
          typeof child === 'object' ? child :
createTextElement(child)
        )
    }
  };
}
```

CORE ENGINE IMPLEMENTATION

Engine Implementation: Recursive renderToDOM & Mounting

Transforming in-memory VNode trees into physical browser DOM elements.

Mounting Protocol

- 1. Node Creation: Use document.createTextNode for TEXT_ELEMENT, document.createElement for HTML tags.
- 2. Prop Assignment: Set attributes (className, id, style) and attach event listeners dynamically.
- 3. Recursive Traversal: Recursively call renderToDOM for all children and append them to parent node.
- 4. Target Mounting: Replace or append the resulting real DOM tree into container using replaceChildren().

render-to-dom.js

```
export function renderToDOM(vNode) {
  // 1. Create text node or standard HTML element
  const dom = vNode.type === 'TEXT_ELEMENT'
    ? document.createTextNode(vNode.props.nodeValue)
    : document.createElement(vNode.type);

  // 2. Assign attributes and listeners
  Object.keys(vNode.props)
    .filter((key) => key !== 'children' && key !== 'nodeValue')
    .forEach((name) => {
      if (name.startsWith('on')) {
        dom.addEventListener(name.toLowerCase().substring(2),
vNode.props[name]);
      } else {
        dom[name === 'className' ? 'className' : name] =
vNode.props[name];
      }
    });

  // 3. Recurse for children
  vNode.props.children.forEach((child) =>
dom.appendChild(renderToDOM(child)));
  return dom;
}
```

ALGORITHMS & COMPLEXITY

Reconciliation Engine: O(n) Heuristic Tree Diffing

How React reduces an NP-hard O(n^3) tree comparison to linear O(n) execution.

The Two Fundamental Heuristics

- Generic tree comparison is O(n^3) — impractical for modern 60fps applications.
- Heuristic 1: Two elements of different types generate entirely different trees. Old tree is dismantled completely.
- Heuristic 2: Child element stability is signaled via unique 'key' props across render passes.
- Diffing Scope: Comparison only occurs at the same hierarchy level (Breadth-first level matching).

diff-logic.js

```
// Diffing Heuristic Decision Tree:
function diff(oldVNode, newVNode) {
  // Rule 1: Type change -> Complete teardown & recreate
  if (!oldVNode || oldVNode.type !== newVNode.type) {
    return { type: 'REPLACE', newVNode };
  }
  // Rule 2: Text node update
  if (newVNode.type === 'TEXT_ELEMENT') {
    return oldVNode.props.nodeValue !==
newVNode.props.nodeValue
      ? { type: 'TEXT', text: newVNode.props.nodeValue }
      : { type: 'NOOP' };
  }
  // Rule 3: Same type -> Diff props & diff children by key
  return {
    type: 'UPDATE',
    props: diffProps(oldVNode.props, newVNode.props),
    children: diffChildren(oldVNode.props.children,
newVNode.props.children)
  };
}
```

ALGORITHMS & COMPLEXITY

Reconciliation: Patch Operations & The Key Prop Contract

Preventing state corruption and minimizing expensive DOM destruction.

The Anatomy of Key Stability

- Patch Action Types: CREATE (mount new node), REMOVE (unmount old node), UPDATE_PROPS, REORDER.
- Array Index Anti-Pattern: Using array indices as keys corrupts local component state on deletion/insertion!
- Stable Identity: Keys must be globally unique within the sibling list and persistent across renders.
- Fiber Concept: React's Fiber architecture links nodes as singly-linked lists (child, sibling, return).

key-anti-pattern.js

```
// ❌ CATASTROPHIC BUG: Index as Key during item removal
// Items: ['Task A', 'Task B'] -> User deletes 'Task A'
// Old: Key 0 ('Task A' [Input State: 'Draft A']), Key 1 ('Task B')
// New: Key 0 ('Task B') -> DOM thinks Key 0 was UPDATED!
// Result: 'Draft A' text remains inside 'Task B' input box!

// ✅ CORRECT CONTRACT: Persistent Entity ID
<ul>
  {tasks.map((task) => (
    <TaskItem key={task.uuid} data={task} />
  ))}
</ul>
```

IN-CLASS SPRINT 1 | 35 MINUTES

Exercise 1: Building Mini-React VNode & Mounting Engine

Deliverable: Zero div-soup, strict typeguard verification, and live DevTools audit.

Task Specifications & Constraints

- Objective: Implement createElement, createTextElement, and renderToDOM from scratch.
- Semantic Constraint: Output must produce semantic tags (<main>, <section>, <button>) — div-soup banned.
- Security Checkpoint: Verify XSS resistance when passing '<script>alert(1)</script>' string as child.
- DevTools Audit: Inspect Elements panel to ensure DOM nodes match VNode tree without orphan nodes.
- Git Requirement: Commit 1: 'feat(core): implement createElement factory' | Commit 2: 'feat(core): implement renderToDOM'.

test-runner.js

```
// Checkpoint 1 Verification Test Suite:
import { createElement, renderToDOM } from './mini-react.js';

const vApp = createElement('main', { id: 'root-view', role:
'main' },
  createElement('header', { className: 'hero' },
    createElement('h1', null, 'Mini React Engine'),
    createElement('p', null, '<img onerror=alert(1)> Safe
Text')
  ),
  createElement('button', { onClick: () => console.log('Ping')
}, 'Click')
);

const root = document.getElementById('app');
root.replaceChildren(renderToDOM(vApp));
console.assert(root.querySelector('button') !== null, 'Mount
Failed');
```

COMPONENT DESIGN

Component Architecture: Pure Functions & Unidirectional Flow

Deconstructing UI into deterministic mathematical functions of props and state.

Functional Component Principles

- Mathematical Purity: f(props, state) = UI representation (VNode tree).
- Capitalization Signal: Compiler differentiates native tags (<header>) from custom components (<Header />).
- Props Immutability: Props flow strictly downward (unidirectional). Children must never mutate received props.
- Composition over Inheritance: Use props.children and render props for flexible, decoupled UI assembly.

functional-component.jsx

```
// Type contract for Component Props
interface CardProps {
  title: string;
  status: 'active' | 'archived';
  children: VNode | VNode[];
}

// Pure Functional Component
export function StatusCard({ title, status, children }:
CardProps) {
  return (
    <article className={`card card--${status}`}>
      <header className="card__header">
        <h3>{title}</h3>
        <span className="badge">{status.toUpperCase()}</span>
      </header>
      <div className="card__body">{children}</div>
    </article>
  );
}
```

REACTIVE STATE ENGINE

Reactive State Under the Hood: Cursor & Closure Memory

Why function components can be completely stateless while preserving state.

The External State Store Model

- The Illusion: Developers think state lives inside the function. In reality, functions re-execute from scratch!
- The Reality: State is stored in an external linear array managed by the framework's runtime dispatcher.
- The Cursor Index: A global integer pointer tracks which hook is currently requesting state during render.
- Closure Power: The returned setState function closes over the specific cursor index allocated during that pass.

state-cursor-concept.js

```
// Framework Internal State Memory Structure:
const stateStore = {
  states: [], // [val1, val2, val3]
  cursor: 0   // Incremented on every hook call
};

function renderPass() {
  stateStore.cursor = 0; // Reset cursor before component
execution
  App();                 // Component reads from stateStore[0],
stateStore[1]...
}

// If conditional execution changes hook order:
// stateStore[0] gets mixed with stateStore[1] -> SYSTEM
CORRUPTION!
```

REACTIVE STATE ENGINE

Deconstructing useState: Dispatcher Queue & Batch Rerender

Complete implementation of the useState hook using closures and render triggers.

Operational Sequence

- 1. Initialization: If states[cursor] is undefined, assign initial value.
- 2. Dispatcher Creation: Bind setter to current cursor value (const currentCursor = cursor).
- 3. Invariant Protection: If setter receives a function, execute updater with previous state.
- 4. Re-render Trigger: Invoke root reconciliation callback when state values actually change (Object.is).

use-state-impl.js

```
let stateStore = [];
let cursor = 0;
let triggerRender = () => {};

export function setRenderer(rerenderFn) { triggerRender =
rerenderFn; }
export function resetCursor() { cursor = 0; }

export function useState(initialValue) {
  const currentCursor = cursor;
  stateStore[currentCursor] = stateStore[currentCursor] ??
initialValue;

  const setState = (nextValue) => {
    const resolvedValue = typeof nextValue === 'function'
      ? nextValue(stateStore[currentCursor])
      : nextValue;
    if (!Object.is(stateStore[currentCursor], resolvedValue)) {
      stateStore[currentCursor] = resolvedValue;
      triggerRender(); // Re-execute component tree
    }
  };

  cursor++;
  return [stateStore[currentCursor], setState];
```

HOOK INVARIANTS

The Invariable Rules of Hooks: Deterministic Call Order

Why hooks cannot be placed inside if-statements, loops, or nested callbacks.

The Mathematical Call-Order Invariant

- Rule 1: Only call hooks at the top level of your component or custom hook.
- Rule 2: Never call hooks inside conditions (if), loops (for/while), or nested functions.
- Failure Mode: Skipping a hook shifts subsequent cursor indices, corrupting all downstream state variables!
- Static Analysis: Enforced at build-time by eslint-plugin-react-hooks.

hook-violation-failure.js

```
// ❌ SEVERE BUG: Conditional Hook Call
function BuggyProfile({ isPremium }) {
  // Render 1: isPremium = true
  // Cursor 0 -> name
  const [name, setName] = useState('Khiem');

  if (isPremium) {
    // Cursor 1 -> tier (Present only in Render 1)
    const [tier, setTier] = useState('Platinum');
  }

  // Render 1: Cursor 2 -> age
  // Render 2 (isPremium = false): Cursor 1 -> age receives
'Platinum'!
  const [age, setAge] = useState(28); // CORRUPTED!
}
```

EVENT ARCHITECTURE

Event Architecture: 1000 Listeners vs Root Delegation Hub

Solving massive memory allocation and garbage collection churn via event bubbling.

Event Delegation Mechanics

- Naive Approach: Attaching 10,000 addEventListener instances consumes megabytes of memory.
- Delegation Solution: Attach a single listener to document or root container for each event type.
- Bubbling Protocol: Event bubbles up DOM tree; root listener inspects event.target.
- Synthetic Wrappers: Cross-browser normalization, event pooling, and polyfills for unified execution.

event-delegation-hub.js

```
export function setupEventDelegation(rootContainer) {
  const supportedEvents = ['click', 'input', 'keydown'];

  supportedEvents.forEach((eventType) => {
    rootContainer.addEventListener(eventType, (nativeEvent) =>
{
      let target = nativeEvent.target;
      const handlerProp =
`on${eventType.charAt(0).toUpperCase() + eventType.slice(1)}`;

      // Traverse up to find virtual event handler
      while (target && target !== rootContainer) {
        if (target.__vnode &&
target.__vnode.props[handlerProp]) {
          target.__vnode.props[handlerProp](nativeEvent);
          break;
        }
        target = target.parentElement;
      }
    });
  });
}
```

IN-CLASS SPRINT 2 | 40 MINUTES

Exercise 2: Reactive State Machine & Delegation Hub

Deliverable: Custom useState with atomic Git history and unidirectional data flow.

Task Specifications & Constraints

- Objective: Build custom useState closure engine and integrate root Event Delegation hub.
- Application: Implement a real-time reactive Task Manager with dynamic filter toggles.
- Git Milestone Commits (Mandatory Atomic Commits):
  - 'feat(state): implement stateStore and resetCursor engine'
  - 'feat(state): implement reactive useState dispatcher'
  - 'feat(events): attach root event delegation listener'
  - 'feat(ui): assemble reactive todo application'
- Checkpoint: Mutate state via UI; verify zero orphan event listeners attached to button children.

checkpoint2-verify.js

```
// Checkpoint 2 Audit:
import { useState, renderApp } from './reactive-engine.js';

function TaskApp() {
  const [tasks, setTasks] = useState(['Review PR', 'Verify
AST']);
  const [filter, setFilter] = useState('ALL');

  return (
    <main className="app-container">
      <header><h2>Tasks: {tasks.length}</h2></header>
      <button onClick={() => setTasks([...tasks, `Task
${Date.now()}`])}>
        Add Task
      </button>
    </main>
  );
}
// Verify: Button click triggers state update and re-render
```

PERFORMANCE ENGINEERING

Performance Engineering: Cascading Re-renders & Diff Overhead

Why child components re-render by default and when it becomes an issue.

The Re-render Propagation Cascade

- Default Behavior: When a parent component renders, JavaScript recursively re-invokes all child components.
- The Myth: Re-rendering does NOT always mean real DOM mutations occur; it re-computes VDOM trees.
- The Bottleneck: For large component trees, allocating thousands of VNode objects exhausts CPU cycles.
- Engineering Rule: Optimize only after profiling with React DevTools Profiler (Never premature optimization).

render-cascade.jsx

```
function ParentDashboard() {
  const [counter, setCounter] = useState(0);

  // Clicking counter causes ParentDashboard to re-render.
  // CONSEQUENCE: <ExpensiveDataVisualization /> is RE-INVOKED,
  // recreating 10,000 VNodes even though its props never
changed!
  return (
    <div>
      <button onClick={() => setCounter(c => c + 1)}>Count:
{counter}</button>
      <ExpensiveDataVisualization dataset={staticLargeDataset}
/>
    </div>
  );
}
```

PERFORMANCE OPTIMIZATION

Component Memoization: Shallow Equality & useMemo Caching

Short-circuiting the reconciliation pipeline using reference comparison.

Memoization Principles

• memo(Component): Performs shallow comparison (Object.is) on oldProps vs newProps. Skips render if identical.

• useMemo(() => fn, [deps]): Caches the return value of expensive pure functions between renders.

• useCallback(fn, [deps]): Preserves function reference stability to prevent breaking child memo wrappers.

• Pitfall: Object/Array literals defined in render body create fresh references every pass, bypassing memo!

memo-implementation.js

```
// Custom Higher-Order Component: memo
export function memo(FunctionalComponent) {
  let prevProps = null;
  let prevVNode = null;

  return function MemoizedComponent(props) {
    if (prevProps && shallowEqual(prevProps, props)) {
      return prevVNode; // Short-circuit: reuse cached VNode
tree
    }
    prevProps = props;
    prevVNode = FunctionalComponent(props);
    return prevVNode;
  };
}

function shallowEqual(objA, objB) {
  const keysA = Object.keys(objA);
  return keysA.length === Object.keys(objB).length &&
    keysA.every(key => Object.is(objA[key], objB[key]));
}
```

HIGH-PERFORMANCE UI

Large-Scale List Optimization: Virtual Scrolling (Windowing)

Maintaining 60fps rendering for 1,000,000 items with constant O(1) DOM footprint.

The Windowing Strategy

- The Problem: Rendering 100,000 <tr> nodes crashes mobile devices and exhausts browser memory (OOM).
- Core Concept: Only render the minimal slice of items currently intersecting the user's viewport (~10-20 items).
- The Phantom Spacer: An invisible container sized to totalHeight (itemCount * itemHeight) simulates full scrollbar.
- Transform Positioning: Rendered DOM nodes are offset dynamically using GPU-accelerated transform: translateY().

virtual-scroll-architecture.html

```
<!-- Virtual Windowing DOM Representation -->
<div class="scroll-viewport" style="overflow-y: auto; height:
500px;">
  <!-- Phantom spacer: 100,000 items * 50px = 5,000,000px -->
  <div class="phantom-spacer" style="height: 5000000px;
position: relative;">
    <!-- Only 12 items mounted in DOM simultaneously -->
    <div class="item-window" style="transform:
translateY(25000px);">
      <div class="row">Item #500</div>
      <div class="row">Item #501</div>
      <!-- ... remaining visible items ... -->
    </div>
  </div>
</div>
```

HIGH-PERFORMANCE UI

Mathematical Model of Windowing: Slice Indices & translateY

Deriving exact viewport slicing formulas for zero-jank high-frequency scrolling.

Calculus of the Viewport Window

- startIndex = Math.max(0, Math.floor(scrollTop / itemHeight) - overscanBuffer)
- endIndex = Math.min(totalCount, Math.ceil((scrollTop + viewportHeight) / itemHeight) + overscanBuffer)
- offsetY = startIndex * itemHeight (applied to item container via transform: translateY)
- Overscan Buffer: 2-3 extra items pre-rendered above and below to prevent blank flashes on fast scrolling.

calculate-window.ts

```
export interface WindowParams {
  scrollTop: number;
  viewportHeight: number;
  itemHeight: number;
  totalCount: number;
  buffer?: number;
}

export function calculateWindow({
  scrollTop, viewportHeight, itemHeight, totalCount, buffer = 2
}: WindowParams) {
  const rawStart = Math.floor(scrollTop / itemHeight);
  const visibleCount = Math.ceil(viewportHeight / itemHeight);

  const startIndex = Math.max(0, rawStart - buffer);
  const endIndex = Math.min(totalCount, rawStart + visibleCount
+ buffer);
  const offsetY = startIndex * itemHeight;
  const totalHeight = totalCount * itemHeight;

  return { startIndex, endIndex, offsetY, totalHeight };
}
```

IN-CLASS SPRINT 3 | 35 MINUTES

Exercise 3: Resilient State Machine & Skeleton Loader

Deliverable: Robust 4-state lifecycle handling (IDLE, LOADING, SUCCESS, ERROR).

Task Specifications & Constraints

- Objective: Eliminate race conditions and inconsistent UI states during async data operations.
- Finite State Machine: Component state strictly restricted to: 'IDLE' | 'LOADING' | 'SUCCESS' | 'ERROR'.
- Skeleton Screen: Render animated pulsing CSS placeholder skeleton during 'LOADING' state.
- Error Boundary & Retry: Render human-readable error with 'Retry Connection' button.
- Git Milestone Commit: 'feat(ui): implement multi-state data component with skeleton feedback'.

state-machine.ts

```
type ViewState<T> =
  | { status: 'IDLE' }
  | { status: 'LOADING' }
  | { status: 'SUCCESS'; data: T }
  | { status: 'ERROR'; error: string };

// Verified Component State Transition:
function DataFeed() {
  const [state, setState] = useState<ViewState<Item[]>>({
status: 'IDLE' });

  const loadData = async () => {
    setState({ status: 'LOADING' });
    try {
      const items = await fetchFeed();
      setState({ status: 'SUCCESS', data: items });
    } catch (err) {
      setState({ status: 'ERROR', error: err.message });
    }
  };
  // Render: Skeleton on LOADING, Data on SUCCESS, Alert on
ERROR
}
```

FRAMEWORK ARCHITECTURE

Paradigm Shift: Runtime VDOM (React) vs Compilers (Svelte/Solid)

Analyzing runtime reconciliation overhead versus build-time code generation.

Architectural Tradeoffs

- Runtime Reconciliation (React / Vue):
  - Ships framework engine (~40-100KB) to browser
  - Compares VDOM trees in memory at runtime
  - Extremely flexible, dynamic UI composition.
- Ahead-Of-Time Compilers (Svelte / Solid):
  - Compiles components directly into surgical DOM mutations (node.data = val)
  - Zero Virtual DOM, zero diffing algorithm runtime
  - Ultra-small bundle sizes, instant execution, fine-grained reactivity.

compiled-output-analogy.js

```
// How Svelte Compiles: count += 1
// ZERO VDOM DIFFING! Direct Surgical Mutation:
function update(changed, ctx) {
  if (changed.count) {
    // Direct browser DOM update instruction
    text_node.data = ctx.count;
  }
}

// React Reconciliation Equivalent:
// 1. Re-run App() -> allocate new VDOM
// 2. Diff old VDOM vs new VDOM -> identify changed text node
// 3. Commit patch to real DOM text node
```

EXAMINATION & DEFENSE

The 3-Minute Live Defense Protocol: Architecture Refactoring

Proof of engineering competence: Live code modification at the evaluation desk.

Evaluation Protocol & Rules

- Format: 1-on-1 live examination at the instructor's desk (60-180 seconds).
- The Mutation: The instructor modifies an architectural variable on your live repository:
  - Swap sync state setter for batch microtask queue
  - Change VNode prop key mapping from 'className' to 'class'
  - Invert virtual window overscan buffer logic.
- Assessment Criteria: Student must diagnose cause, refactor logic live, and verify in DevTools.
- Grading Impact: 25% of overall lab grade. Unverified code yields zero defense score.

live-challenge-examples.sh

```
# Common Live Defense Mutation Challenges:
# Challenge A: "Refactor renderToDOM to support SVG elements with correct namespace."
# -> Fix:
document.createElementNS('http://www.w3.org/2000/svg', type)

# Challenge B: "Your useState doesn't handle function updaters: setCount(c => c + 1). Fix it."
# -> Fix: typeof nextValue === 'function' ? nextValue(prev) : nextValue

# Challenge C: "Inject a cycle-detection check in createElement children flattening."
```

SECURITY & CODE INTEGRITY

AI Failure Mode Audit: Detecting Hallucinations & Security Holes

Mandatory 15% evaluation deliverable: Documenting and refactoring AI risks.

Top 3 Classic AI Failure Modes in UI Development

- 1. Security: AI defaults to el.innerHTML = props.text, opening catastrophic Cross-Site Scripting (XSS).
- 2. Memory Leaks: AI binds event listeners directly inside component render body without unmounting/delegation.
- 3. Phantom Keys: AI sets key={Math.random()} or key={index}, triggering total DOM destruction every render pass.
- Deliverable: AI_FAILURE_AUDIT.md documenting 3 detected bugs with DevTools proof and manual fixes.

AI_FAILURE_AUDIT.md

```
# Sample Audit Report Entry:
## Vulnerability 01: Raw innerHTML Ingestion
- AI Prompt: 'Generate a rich text renderer for comment items.'
- AI Generated Code: container.innerHTML = comment.text;
- Risk: Stored XSS via <img src=x onerror=stealToken()>
- Detection Method: DevTools Security Panel & Git Diff
- Engineering Refactor:
  const textNode = document.createTextNode(comment.text);
  container.replaceChildren(textNode);
```

HOMEWORK 1 | 1 WEEK

HW 1: Production Enterprise Data-Grid (WCAG 2.2 AA)

Industrial-strength accessible component system with 100 Lighthouse score.

Project Specification & Criteria

- Component: Full-featured accessible Data-Grid with sorting, filtering, and pagination.
- Accessibility Contract: WCAG 2.2 Level AA compliance, 4.5:1 color contrast, full screen reader support.
- Keyboard Navigation: Up/Down/Left/Right arrow keys navigate grid cells; Enter selects, Space toggles.
- Lighthouse Audit: Must achieve verified 100/100 score on Accessibility and Best Practices.
- Zero div-soup: Must use native <table>, <thead>, <tbody>, <th scope='col'>, <button>.

data-grid-contract.ts

```ts
export interface ColumnDef<T> {
  header: string;
  accessorKey: keyof T;
  sortable?: boolean;
}

export interface DataGridProps<T> {
  data: T[];
  columns: ColumnDef<T>[];
  onRowSelect?: (item: T) => void;
  ariaLabel: string;
}
// Invariant: Full ARIA grid semantics (role="grid",
role="row", role="gridcell")
```

HOMEWORK 2 | 1 WEEK

HW 2: Concurrent FIFO Audio Visualizer Queue Engine

Advanced async concurrency, audio frequency analysis, and zero memory leaks.

Project Specification & Criteria

- Engine: Web Audio API integration with Web Audio AnalyserNode (FFT 256/512 bins).
- Concurrency Controller: FIFO Queue handling simultaneous track downloads and audio buffer decoding.
- Memory Safety: Explicit cleanup of AudioBufferSourceNodes, cancellation tokens via AbortController.
- Performance: 60fps Canvas or SVG visualizer driven by requestAnimationFrame without state thrashing.

audio-queue-controller.ts

```
export class AudioTaskQueue {
  private queue: AudioTask[] = [];
  private isProcessing = false;
  private activeController: AbortController | null = null;

  async enqueue(trackUrl: string): Promise<AudioBuffer> {
    // Concurrency control with AbortSignal cancellation
    const controller = new AbortController();
    return new Promise((resolve, reject) => {
      this.queue.push({ trackUrl, controller, resolve, reject
});
      this.processNext();
    });
  }
}
```

HOMEWORK 3 | 1 WEEK

HW 3: High-Concurrency Flash-Sale E-Commerce Engine

Realistic Vietnam market scenario: Zero-drift countdown timer & inventory state machine.

Project Specification & Criteria

- Domain: Vietnam Mega-Sale (Shopee / Lazada 11.11 / Midnight Flash Sale simulation).
- Zero-Drift Timer: Accurate millisecond countdown using Date.now() delta vs drift-prone setInterval.
- Optimistic Inventory Lock: Finite state machine handling inventory reservation, lock expiry, and checkout.
- AI Failure Mode Audit: Mandatory AI_FAILURE_AUDIT.md report documenting >= 3 AI flaws.
- Atomic Commits: Minimum 8 discrete commits adhering to Conventional Commits specification.

anti-drift-timer.ts

```
// Precision Anti-Drift Countdown Implementation
export function createAntiDriftTimer(durationMs: number,
onTick: (remaining: number) => void) {
  const targetTime = Date.now() + durationMs;
  let rafId: number;

  function tick() {
    const remaining = Math.max(0, targetTime - Date.now());
    onTick(remaining);
    if (remaining > 0) {
      rafId = requestAnimationFrame(tick); // Zero drift
compared to setInterval
    }
  }
  rafId = requestAnimationFrame(tick);
  return () => cancelAnimationFrame(rafId);
}
```